

Assignment 2 (grupal part)

Instructions in English provided below

50% out of 100% of lab part.

Enunciado (Español)

Introducción

En esta práctica aprenderemos sobre herramientas de medición, fiabilidad de software, mantenimiento y mejora de la calidad del mismo.

Trabajaremos de nuevo con Python y Visual Studio Code. Para la primera parte usaremos la herramienta de Radon para extraer distintas métricas vistas en teoría del software que vamos a analizar. Esto nos servirá para identificar los problemas de software futuros. Para la segunda parte de la práctica, este software estará corriendo en un servidor, cuyo comportamiento deberemos de analizar desde un punto de vista de fiabilidad con los distintos indicadores de rendimiento vistos en teoría. Por último, en la tercera parte de la práctica aprenderemos sobre el mantenimiento y la mejora de calidad de este software, basándonos en las métricas obtenidas en la primera parte y realizando una modificación de código para conseguir una mejora de estas métricas.

Tareas a desarrollar

Parte 1: Herramienta de medición

Se descargará un proyecto de GitHub y se analizará con la herramienta de Radon para obtener diversas métricas:

- 1) Descarga o clona el proyecto GildedRose-Refactoring-Kata (<https://github.com/emilybache/GildedRose-Refactoring-Kata>), en la carpeta de python encontrarás el código en Python. Los dos archivos importantes de código son "gilded_rose.py" y "texttest_fixture.py". Este código es un sistema de inventario de una tienda objetos fantásticos. El primer archivo es la lógica del valor de esos objetos y el segundo es la ejecución de la simulación del valor de los objetos cuando pasen los días definidos (se ejecuta con `python texttest_fixture.py 10` donde el número representa el número de días a simular, en este caso 10). Según pasen los días, los objetos irán aumentando su valor o disminuyendo su inventario, según la lógica programada en "gilded_rose.py". Ejecuta el código y observa la salida para entenderlo.
- 2) Instala la herramienta Radon con pip: `pip install radon`. También existe una extensión en VisualStudio. Consulta la documentación (<https://radon.readthedocs.io/en/latest>) para ver qué métricas puede sacar y qué comandos se deben de utilizar para sacarlas. Un ejemplo de uso es: `radon cc .` lo que calcula la complejidad ciclomática para todos los archivos presentes en el directorio.

Documenta a continuación en tu práctica:

- a. Analiza todas las métricas que consideres relevantes proporcionadas por Radon. Explica en base de resultados de estas métricas los resultados del software y los

problemas encontrados, riesgos y conclusiones que puedes obtener. Es muy importante interpretar y justificar los resultados obtenidos.

Parte 2: Fiabilidad del sistema

Esta aplicación se ha desplegado en un servidor propenso a fallos. Se pide:

- 1) Crear un fichero Excel que permita calcular automáticamente los indicadores básicos de fiabilidad (p.ej., número de fallos, MTBF, MTTF, MTTR, probabilidad de fallo, etc.) cuando se introduce la muestra, a lo largo del tiempo (al menos, para 20 unidades de tiempo), de los períodos de *uptime* y *downtime* de un sistema (ver un ejemplo mínimo en Figura 1). Se valorará:
 - a. La representación gráfica (lo más desarrollada posible) de los períodos de funcionamiento y de fallo. Mirar la Figura 1 como guía.
 - b. La posibilidad de calcular indicadores también mediante la introducción de los valores de período de tiempo de control de fiabilidad (por ejemplo, meses, semanas concretas, días, horas, segundos, etc.) y la indicación de una serie de períodos concretos de caída del sistema. Mirar Figura 2 como guía.
 - c. Curva de fiabilidad para una probabilidad de supervivencia del sistema a lo largo de 10 semanas. Recuerda que la fórmula viene dada por:

$$R(t) = e^{-\lambda \cdot t}$$

donde lambda es la probabilidad obtenida y el tiempo es el tiempo dado.
Mirar Figura 3 como guía.

Documenta a continuación en tu práctica:

- a. Resumen de cómo has implementado el Excel.

Parte 3: Mantenimiento y mejora de calidad

Partiendo de los resultados obtenidos en la parte 1:

- 1) Analiza qué tipo de mejora y en qué parte de software podrías hacer para reducir la complejidad obtenida en la parte 1.
- 2) Modifica el código para realizar la mejora.
- 3) Compara los resultados obtenidos con Radon en la parte 1 y parte 3.

Documenta a continuación en tu práctica:

- a. Explica los cambios en el código que has realizado y por qué
- b. Compara los resultados obtenidos con Radon y explica las diferencias que has encontrado, justificando tu respuesta.

Assignment (English)

Introduction

In this lab assignment, we will focus on measurement tools, software reliability, maintenance strategies, and quality improvement techniques.

We will continue working within the Python ecosystem using Visual Studio Code.

- In Part 1, we will employ the Radon tool to extract and analyze various software metrics covered in the theoretical lectures. This analysis will enable us to identify potential technical debt and future software issues.
- In Part 2, we will simulate the software running in a server environment to analyze its behavior from a reliability standpoint, calculating the key performance indicators (KPIs) discussed in class.
- Finally, Part 3 focuses on software maintenance and quality improvement. Guided by the metrics obtained in Part 1, we will perform a code refactoring to optimize the software's design and enhance these metric scores.

Tasks to perform

Part 1: Software Measurement Tools

In this section, a GitHub project will be downloaded and analyzed using the Radon tool to obtain various software metrics:

- 1) Download or clone the GildedRose-Refactoring-Kata project (<https://github.com/emilybache/GildedRose-Refactoring-Kata>), in the python folder you will find the code in Python. The two important code files are "gilded_rose.py" and "texttest_fixture.py". This code is an inventory system for a fantasy item shop. The first file is the logic of the value of those items and the second is the execution of the simulation of the value of the items as the defined days pass (it is executed with `python texttest_fixture.py 10` where the number represents the number of days to simulate, in this case 10). As the days pass, the items will increase their value or decrease their inventory, according to the logic programmed in "gilded_rose.py". Execute the code and observe the output to understand it.
- 2) Install the Radon tool with pip: `pip install radon`. There is also an extension in VisualStudio. Consult the documentation (<https://radon.readthedocs.io/en/latest>) to see what metrics it can extract and what commands must be used to extract them. A usage example is: `radon cc` . which calculates the cyclomatic complexity for all files present in the directory. Document below in your assignment: a. Analyze all the metrics you consider relevant provided by Radon. Explain based on the results of these metrics the software results and the problems found, risks and conclusions you can obtain. It is very important to interpret and justify the results obtained.

Document the following in your practice report:

- a. Analyse all the metrics provided by Radon that you consider relevant. Based on the results of these metrics, explain the software's results and the problems encountered, risks and conclusions you can draw. It is very important to interpret and justify the results obtained.

Part 2: System reliability

This application has been deployed on a server prone to failure. Perform the following tasks:

- 1) Create an Excel file that automatically calculates basic reliability indicators (e.g., number of failures, MTBF, MTTF, MTTR, failure probability, etc.) when the sample is entered, over time (at least for 20 time units), of the uptime and downtime periods of a system (see a minimal example in Figure 1). The following will be assessed:

- The graphical representation (as detailed as possible) of the periods of operation and failure. See Figure 1 for guidance.
- The possibility of calculating indicators also by entering the reliability control time period values (e.g., months, specific weeks, days, hours, seconds, etc.) and indicating a series of specific system downtime periods. See Figure 2 for guidance.
- Reliability curve for a probability of system survival over 10 weeks. Remember that the formula is given by:

$$R(t) = e^{-\lambda \cdot t}$$

where lambda is the probability obtained and time is the given time. See Figure 3 for guidance.

Document the following in your practice report:

- Summary of how you have implemented Excel.

Part 3: Quality maintenance and improvement

Based on the results obtained in part 1:

- Analyse what type of improvement you could make and in which part of the software to reduce the complexity obtained in part 1.
- Modify the code to make the improvement.
- Compare the results obtained with Radon in part 1 and part 3.

Document the following in your practice report:

- Explain the changes you have made to the code and why.
- Compare the results obtained with Radon and explain the differences you have found, justifying your answer.

Anexo / Annex

Figura 1. Ejemplo de cálculos de fiabilidad:

Figure 1. Example of reliability calculations:

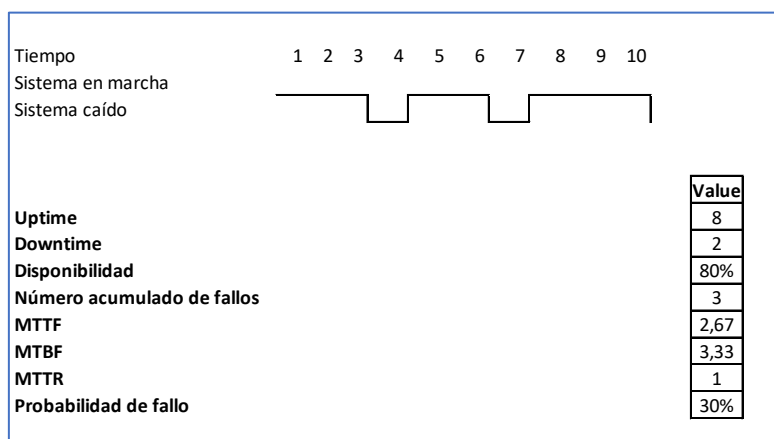


Figura 2. Ejemplo de cálculos de fiabilidad dado un periodo de tiempo (10 unidades):

Figure 2. Example of reliability calculations given a period of time (10 units):

Test period	62	weeks
Specify the value type		
Downtime 1	1	hours
Downtime 2	300	minutes
Downtime 3	2	days
Downtime 4	50	weeks
Downtime 5	20	minutes
Downtime 6	24	hours
Downtime 7	3	minutes
Downtime 8	2	days
Downtime 9	2	weeks
Downtime 10		minutes
Value in:		days
Uptime	412,00	days
Downtime	404,00	days
Availability	82,00%	
Number of failures	9	
MTTF	23,00	days
MTBF	6,89	days
MTTR	44,89	days
Probability of failure	14,52%	

Figura 3. Ejemplo de curva de fiabilidad dado un periodo de tiempo (300 horas):

Figure 3. Example of a reliability curve given a period of time (300 hours):

